Real-Time Audio

Intro to Audio Tech 2
Jocelyn Kavanagh
Thursday, April 2

Georgia Tech

Lesson Outline

- Real-time systems
- Block-based processing and buffering

Real-Time Systems

Real-Time Audio

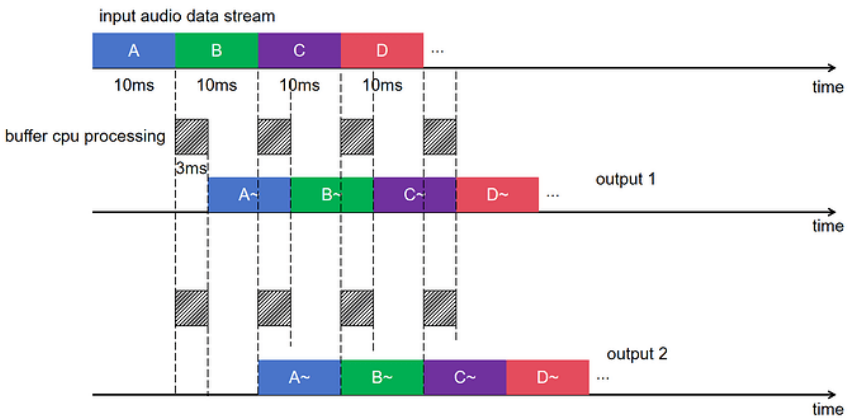
- In Python, we have generated (or read through) all of our sounds in advanced (offline)
- In real-time systems, we continuously process audio as it comes (or as it is needed)
- When we play and record audio, a constant stream of data is going to and from your sound device
- The rate of those data values (or samples) is already defined
  - Sampling rate
- Real-time doesn't change the DSP we've learned, we just have to apply it strategically
  - The only DSP constraint in real-time is our algorithms cannot depend on future values

Building for Real-Time

- We must do all of our computation and processing in a fixed period of time
- We need to know how fast we can process a given signal to move onto the next
- Our output must perform at a fast enough rate to keep up with the changing input
- Typically, we stream audio data in blocks of samples at a time

Audio Buffering

- Instead of sending one sample at a time, systems typically group samples into chunks or blocks
- This allows us to do DSP over more than one sample and helps us be more efficient in data transmission
- Modern systems can have block sizes of just a few samples, but you'll typically see ranges from 32-4096



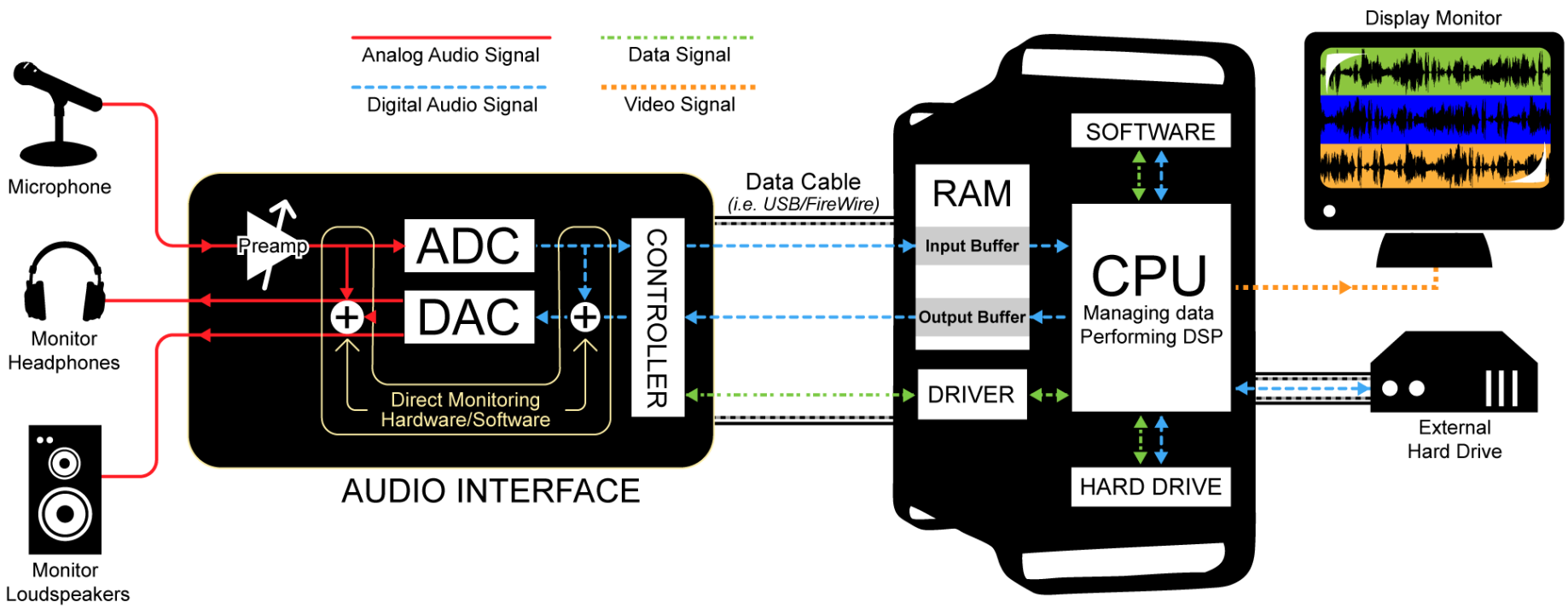
Example: Digital Mixer

We are running a digital mixer at a sampling rate of 48kHz and buffer size of 480 samples

1. A mic connects and produces a continuous analog audio signal
2. That analog signal is converted to digital by the ADC at a fixed rate (48kHz)
3. The ADC will fill an input buffer (480 samples) with the newly converted signal
4. When the buffer is full, we start DSP (gain, delay, compressor, etc)
5. The processed buffer is sent to the DAC which converts back to analog

- This happens continuously at a very fast rate!
- The processed buffer must be sent to the DAC by the time the old buffer has been played
  - Less than the buffer size / sampling rate (in this case 10ms!)

Example: Digital Mixer



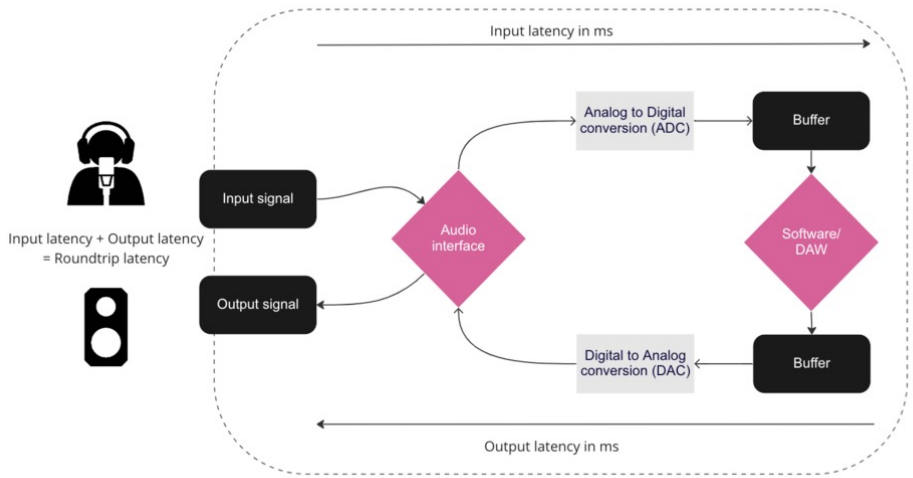
Block Sizes

- Why might we want a larger buffer size?
  - We can do more processing!
- Why might we not want a larger buffer size?
  - Latency!

- Block size determines latency and interactivity
- Larger block sizes are more forgiving with processing time, but they come with more latency
- If we are late processing our data, we will have buffer underrun
  - Glitches, pops, silence, etc

Latency

- Delay between the stimulus and response
- The amount of processing time needed can be from the algorithm itself or from the interface (block size, actual hardware conversion, etc)
- For highly interactive systems like monitoring, you want latency less than ~5ms but under ~10ms is usually safe for other applications



Real-Time Constraints

- Performance
  - Processing time must be less than than the block size / sampling rate (for every buffer!)
- Causality
  - We have no knowledge of future samples so we can’t depend on them
- Latency
  - Time between input and system response

Is it possible in real time?

- Delay based effects?
  - Yes
- EQ and filtering?
  - Yes
- Compression and dynamic processing?
  - Yes
- Reversing Audio?
  - No
- FFT?
  - Yes (depending on block size with tradeoffs)
- Read an IR from a file and convolve?
  - No (read the file offline then yes)
- BPM detection?
  - No (and yes?)
- Normalization across file?
  - No

Let’s Build a Sine Generator

- Open the ProJucer
- File -> New Project
- Choose Plug-in -> basic then name your project and hit Create Project
- Go to modules and make sure ProJucer knows the path to your JUCE modules
- Go to exporters and make sure you are building to at least your IDE
- Select the correct exporter at the top and hit the button to open the project in your IDE
- Build and run the project in your IDE